

LAPORAN TUGAS

**WEB SERVER
(TIF 330805)**

MINGGU KE 4



ACARA 8

DIBUAT OLEH:

NAMA: CINTYA AZWA SAFRINA R.H

NIM:E41250631

GOL:A

**PROGAM STUDY DILUAR KAMPUS UTAMA(PSDKU) POLITEHNIK NEGRI JEMBR
TEHNIK INFORMATIKA KAB.NGANJUK JL. JENDRAL GATOT SUBROTO NO
106,KAUMAN,KECAMATAN NGANJUK,KABUPATEN NGANJUK JURUSAN
TEKNOLOGI INFORMASI POLITEHNIK NEGRI JEMBER TAHUN 2026**

PENDAHULUAN

Pada praktikum Acara 7, aplikasi telah mampu membaca data mahasiswa dari database menggunakan operasi dasar (*read-only*). Namun, sebuah sistem informasi akademik yang fungsional membutuhkan lebih dari sekadar menampilkan data — pengguna juga harus dapat menambah, mengubah, dan menghapus data sesuai kebutuhan. Kebutuhan inilah yang mendasari implementasi CRUD (Create, Read, Update, Delete) secara lengkap pada praktikum ini.

Dalam berinteraksi dengan database, terdapat risiko keamanan yang serius apabila query SQL dibangun dengan cara menggabungkan (*concatenate*) input pengguna secara langsung ke dalam string SQL. Praktik ini membuka celah SQL Injection, yaitu serangan di mana pengguna memasukkan potongan kode SQL berbahaya melalui form input, yang kemudian dieksekusi sebagai bagian dari query. Untuk mencegah hal tersebut, digunakan prepared statement, yaitu teknik yang memisahkan struktur query SQL dari data yang dimasukkan pengguna. Dengan prepared statement, input pengguna diperlakukan murni sebagai data, bukan sebagai bagian dari perintah SQL, sehingga serangan seperti ' OR '1'='1 tidak akan berhasil membocorkan data.

Untuk mengelola koneksi database secara efisien, digunakan pola desain Singleton pada class Database, yang memastikan hanya ada satu instance koneksi PDO yang digunakan di sepanjang aplikasi, mencegah pembuatan koneksi berulang yang tidak perlu. Selain itu, praktikum ini juga menerapkan relasi One-to-Many antar tabel (satu program studi memiliki banyak mahasiswa dan banyak mata kuliah), yang ditampilkan menggunakan perintah JOIN agar data lebih informatif.

IMPLEMENTASI

Langkah Kerja

1. Membuat class Database sebagai *singleton*, yang mengelola satu-satunya instance koneksi PDO sepanjang aplikasi berjalan, dengan konfigurasi ATTR_ERRMODE dan ATTR_EMULATE_PREPARES diaktifkan sesuai anjuran modul.

Run SQL query/queries on database si_akademik_acara7:

```
1 INSERT INTO mahasiswa (nim, nama, email, prodi_id, angkatan, status) VALUES
2 ('2403001', 'Doni Firmansyah', 'doni@email.com', 3, 2024, 'aktif'),
3 ('2301001', 'Eka Putri', 'eka@email.com', 1, 2023, 'cuti'),
4 ('2201001', 'Fajar Nugroho', 'fajar@email.com', 2, 2022, 'lulus');
```

1. Membuat MahasiswaModel, ProdiModel, dan MatakuliahModel, masing-masing dengan method all(), find(), create(), update(), dan delete() menggunakan *prepared statement*.
2. Membuat MahasiswaController, ProdiController, dan MatakuliahController yang menangani seluruh alur CRUD: menampilkan form, memvalidasi input sederhana, dan memanggil Model yang sesuai.
3. Membuat View untuk setiap resource: halaman daftar (index), form tambah (create), dan form edit (edit), menggunakan Bootstrap.
4. Menampilkan nama program studi pada tabel Mahasiswa dan Mata Kuliah menggunakan JOIN ke tabel prodi, sesuai relasi *One-to-Many*.
5. Menambahkan atribut onsubmit="return confirm(...)" pada form hapus, sehingga pengguna harus mengonfirmasi terlebih dahulu sebelum data benar-benar dihapus.
6. Memastikan seluruh operasi store, update, dan destroy menggunakan method **POST**, sesuai *best practice* yang disebutkan pada modul.
7. **Tugas Mandiri:** menambahkan fitur pencarian pada halaman index Mahasiswa berdasarkan nama atau NIM, menggunakan klausa LIKE dengan *prepared statement*.
8. Menguji seluruh alur CRUD untuk ketiga resource, termasuk pengujian fitur pencarian.

Struktur Kode

Database class dengan pola Singleton (app/Core/Database.php)

```
class Database
{
    private static ?PDO $instance = null;

    public static function getInstance(): PDO
    {
        if (self::$instance === null) {
            $config = require __DIR__ . '/../../config/database.php';
            $dsn =
                "mysql:host={$config['host']};dbname={$config['dbname']};charset={$config['charset']}";

            self::$instance = new PDO($dsn, $config['username'], $config['password'], [
                PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
                PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
                PDO::ATTR_EMULATE_PREPARES => false,
            ]);
        }
        return self::$instance;
    }
}
```

```
public function create(array $data): void
{
    $pdo = Database::getInstance();
    $stmt = $pdo->prepare(
        "INSERT INTO mahasiswa (nim, nama, email, prodi_id, angkatan, status)
        VALUES (:nim, :nama, :email, :prodi_id, :angkatan, :status)"
    );
    $stmt->execute([
        'nim' => $data['nim'],
        'nama' => $data['nama'],
        'email' => $data['email'],
        'prodi_id' => $data['prodi_id'],
        'angkatan' => $data['angkatan'],
        'status' => $data['status'],
    ]);
}
```

Tugas Mandiri: pencarian dengan LIKE (MahasiswaModel::all())

```
public function all(?string $keyword = null): array
{
    $pdo = Database::getInstance();
    $sql = "SELECT mahasiswa.*, prodi.nama AS prodi_nama
            FROM mahasiswa JOIN prodi ON mahasiswa.prodi_id =
            prodi.id";
```

```

        if ($keyword !== null && $keyword !== '') {
            $sql .= " WHERE mahasiswa.nama LIKE :kw1 OR
mahasiswa.nim LIKE :kw2";
        }
        $sql .= " ORDER BY mahasiswa.id ASC";

        $stmt = $pdo->prepare($sql);
        if ($keyword !== null && $keyword !== '') {
            $stmt->execute(['kw1' => '%' . $keyword . '%', 'kw2'
=> '%' . $keyword . '%']);
        } else {
            $stmt->execute();
        }
        return $stmt->fetchAll();
    }
}

```

Konfirmasi JavaScript sebelum hapus (View index)

```

<form action="mahasiswa/5/delete" method="post"
    onsubmit="return confirm('Yakin ingin menghapus data ini?');">
    <button type="submit" class="btn btn-sm btn-outline-danger">Hapus</button>
</form>

```

Hasil Tampilan



Politeknik Negeri Jember
Sistem Informasi Akademik

Login

Username

Password

Login



Sistem Informasi Akademik
Dashboard

Selamat datang, Admin.

Selamat datang, **admin**.

Mahasiswa Logout



Sistem Informasi Akademik

Daftar Mahasiswa

Cari nama atau NIM...

Cari

+ Tambah Mahasiswa

NIM	Nama	Email	Prodi	Angkatan	Status	Aksi
2401001	Budi Santoso	budi@email.com	Teknik Informatika	2024	aktif	Edit Hapus
2401002	Ani Wijaya	ani@email.com	Teknik Informatika	2024	aktif	Edit Hapus
2402001	Citra Lestari	citra@email.com	Sistem Informasi	2024	aktif	Edit Hapus
2403001	Doni Firmansyah	doni@email.com	Teknik Komputer	2024	aktif	Edit Hapus
2301001	Eka Putri	eka@email.com	Teknik Informatika	2023	cuti	Edit Hapus
2201001	Fajar Nugroho	fajar@email.com	Sistem Informasi	2022	lulus	Edit Hapus

[Kembali ke Dashboard](#)



Sistem Informasi Akademik

Daftar Mahasiswa

budi

Cari

Reset

+ Tambah Mahasiswa

NIM	Nama	Email	Prodi	Angkatan	Status	Aksi
2401001	Budi Santoso	budi@email.com	Teknik Informatika	2024	aktif	Edit Hapus

[Kembali ke Dashboard](#)

**Sistem Informasi Akademik**
Edit Mahasiswa

NIM

Nama

Email

Program Studi

Angkatan

Status

Update

Batal

**Sistem Informasi Akademik**
Tambah Mahasiswa

NIM

Nama

Email

Program Studi

Angkatan

Status

Simpan

Batal

Analisis

Hasil pengujian membuktikan bahwa fitur pencarian berhasil memfilter data secara akurat. Ketika kata kunci "budi" dimasukkan, sistem hanya menampilkan satu baris data ("Budi Santoso") dari total enam data yang ada, membuktikan bahwa klausa LIKE pada query berhasil mencocokkan sebagian teks pada kolom nama atau nim. Pada implementasi awal, sempat terjadi error "*Invalid parameter number*" karena parameter bernama :keyword digunakan dua kali dalam satu query, sementara koneksi PDO dikonfigurasi dengan `ATTR_EMULATE_PREPARES => false` (native prepared statement). Konfigurasi ini memang direkomendasikan modul karena lebih aman dan sesuai standar, namun memiliki konsekuensi bahwa driver MySQL secara native tidak mengizinkan penggunaan nama parameter yang sama secara berulang. Masalah ini diselesaikan dengan memberikan nama parameter yang berbeda (:kw1 dan :kw2) untuk masing-masing kondisi pencarian.

Penerapan pola *Singleton* pada class Database terbukti efisien, karena seluruh Model (MahasiswaModel, ProdiModel, MatakuliahModel) dapat menggunakan koneksi database yang sama tanpa perlu membuat koneksi baru setiap kali sebuah Model diinstansiasi, mengurangi beban pada server database.

Penggunaan JOIN pada method all() di MahasiswaModel dan MatakuliahModel berhasil menampilkan nama program studi secara langsung (misalnya "Teknik Informatika"), bukan hanya angka prodi_id, sehingga data lebih mudah dipahami oleh pengguna akhir. Hal ini membuktikan relasi *One-to-Many* antara tabel prodi dengan mahasiswa maupun matakuliah berfungsi dengan baik.

Seluruh operasi yang mengubah data (store, update, destroy) berhasil diimplementasikan menggunakan method POST, sesuai *best practice* yang dianjurkan modul, untuk mencegah operasi pengubahan data dilakukan secara tidak sengaja melalui GET (misalnya melalui *bookmark* atau *crawler*). Konfirmasi JavaScript (confirm()) pada form hapus juga berhasil mencegah penghapusan data yang tidak disengaja, memberikan kesempatan terakhir bagi pengguna untuk membatalkan aksi tersebut.

Secara keseluruhan, praktikum ini berhasil menghasilkan aplikasi dengan operasi data yang lengkap dan aman, menggabungkan seluruh konsep yang telah dipelajari sejak Acara 1 hingga Acara 8 — mulai dari arsitektur client-server, MVC, routing, middleware autentikasi, hingga integrasi database dengan CRUD penuh.

KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa:

1. *Prepared statement* efektif mencegah serangan SQL Injection dengan memisahkan struktur query dari data yang dimasukkan pengguna, sehingga input berbahaya diperlakukan sebagai data biasa, bukan perintah SQL.
2. Pola *Singleton* pada class Database memastikan aplikasi hanya menggunakan satu koneksi PDO yang digunakan bersama oleh seluruh Model, meningkatkan efisiensi penggunaan sumber daya.
3. Operasi CRUD lengkap (Create, Read, Update, Delete) dapat diimplementasikan secara konsisten untuk berbagai resource (Mahasiswa, Prodi, Mata Kuliah) dengan pola Model-Controller-View yang seragam.
4. Penggunaan JOIN memungkinkan penampilan data yang lebih informatif dengan menggabungkan data dari tabel yang saling berelasi, tanpa perlu mengubah struktur data pada tabel utama.
5. Konfigurasi PDO::ATTR_EMULATE_PREPARES => false memberikan keamanan yang lebih baik (native prepared statement), namun mengharuskan pengembang menggunakan nama parameter yang unik meskipun kondisi query serupa.
6. Konfirmasi melalui JavaScript sebelum operasi hapus merupakan langkah penting untuk mencegah kehilangan data secara tidak sengaja.
7. Fitur pencarian menggunakan LIKE dengan *prepared statement* memungkinkan pengguna menemukan data spesifik dengan cepat tanpa mengorbankan keamanan aplikasi.